

Naegling: Um Sistema para Implementação e Administração de *Clusters* e Submissão de Tarefas

Daniel Yokoyama², Henrique Klôh³, Bruno Schulze¹, Antonio Mury¹

¹ComCiDiS - Laboratório Nacional de Computação Científica - LNCC
Petrópolis-RJ - Brasil

²Laboratório Nacional de Computação Científica - LNCC
Petrópolis-RJ - Brasil

³Faculdade de Educação Tecnológica do Estado do Rio de Janeiro - FAETERJ
Petrópolis-RJ - Brasil

Abstract. *The configuration process and deployment of a cluster can be time consuming and technically complex. Allied to this problem, it is common for computational resources to be dedicated to running applications that do not use all the available resource in the best possible way rendering it underutilized. Given these assumptions, this paper presents a tool to create and manage virtual clusters to facilitate its configuration and administration, as well as provide its users with a simplified interface for submitting jobs. This proposal seeks to automate the process of creating and managing clusters, simplify the process of submitting applications and enable the creation of virtual clusters on demand for different purposes, maximizing the efficiency of resource usage.*

Resumo. *O processo de configuração e disponibilização de um cluster pode ser demorado e tecnicamente complexo. Aliado a este problema, é comum que recursos computacionais fiquem dedicados para execução de aplicações que não utilizam o recurso da melhor forma tornando-o subutilizado. Considerando essas premissas, este trabalho apresenta uma ferramenta capaz de criar e gerenciar clusters virtuais visando facilitar a sua configuração e administração, bem como fornecer aos seus usuários uma interface simplificada para a submissão de tarefas. Esta proposta procura automatizar o processo de criação e gerência de clusters, simplificar o processo de submissão de aplicações e possibilitar a criação de clusters virtuais sob-demanda para diferentes fins, maximizando a eficiência da utilização dos recursos.*

1. Introdução

Um *cluster* computacional consiste na agregação de vários servidores com funcionamento e capacidade de processamento independente em uma única entidade. Caracteriza-se por possuir capacidade de processamento e disponibilidade de recursos muito superiores aos módulos que a formam.

O seu uso para a solução de problemas complexos, torna-se cada dia mais comum e necessário, principalmente no cenário de CMPD (Computação Massivamente Paralela e Distribuída), permitindo a resolução de tarefas que anteriormente seriam inviáveis devido à grande quantidade de cálculos necessários para obter o resultado no tempo desejado.

Por este motivo, diversas áreas, tanto do setor acadêmico como produtivo, cada vez mais empenham-se na modelagem e paralelização de seus algoritmos. Isso permite que o processamento de uma tarefa ocorra em diversos servidores, tornando-se uma ferramenta para setores como biomedicina, a meteorologia, a astronomia, entre outros.

Entretanto, o processo de configuração e disponibilização de um *cluster* pode ser tecnicamente demorado e complexo. Necessita de pessoal capacitado para mantê-lo e configurá-lo para atender os requisitos do usuário[Pacheco 2011]. As restrições anteriormente citadas aliadas ao fato de que único ambiente não é capaz de atender a todas as necessidades dos usuários, poderá acarretar no retardo ou mesmo na impossibilidade da execução dos seus trabalhos.

Este trabalho tem como objetivos principais: 1) criação, configuração e gerência de *clusters* virtuais; 2) auxiliar os usuários na submissão e gerência de suas aplicações nos clusters, de modo a garantir a transparência da utilização dos recursos; e 3) gerenciar a criação e configuração de *clusters* virtuais permitindo ainda a submissão de aplicações de forma a auxiliar o usuário menos experiente na utilização dos recursos.

O trabalho a seguir está organizado da seguinte forma: a seção 2 apresenta trabalhos relacionados com o processo de criação, configuração e gerência de *clusters*; a seção 3 descreve as funcionalidades e a arquitetura da ferramenta apresentada neste trabalho e a seção 5 apresenta as conclusões e os trabalhos futuros.

2. Trabalhos Relacionados

A criação e configuração de *clusters* é, muitas vezes, uma tarefa onerosa devido a mão-de-obra e tempo necessários para configuração de todos os nós do *cluster*. Para auxiliar e facilitar esta tarefa, dentro do contexto deste trabalho, foram criadas algumas ferramentas que contribuem para gerência e utilização do *cluster*. A seguir serão apresentadas algumas destas ferramentas.

O *PelicanHPC* [PELICANHPC 2012] fornece uma imagem em ISO para CD e USB com um sistema operacional GNU/Linux, permitindo a rápida configuração de um ambiente de *cluster* voltado para o processamento de tarefas desenvolvidas com a biblioteca MPI. Funciona utilizando pelo menos um computador que irá iniciar utilizando a imagem ISO disponível. Para adicionar nós a este *cluster* o *PelicanHPC* utiliza o PXE (*Preboot Execution Environment*). Desta forma, a máquina que é iniciada com a imagem ISO atua como um servidor PXE, através do qual os outros nós do *cluster* receberão o sistema operacional.

O *Rocks Cluster* [ROCKSCLUSTER 2012] é uma distribuição GNU/Linux voltada para a criação *clusters*. Para seu uso o *Rocks* é instalado em um servidor, que se tornará o nó master, e a partir deste todos os nós de trabalho serão criados automaticamente utilizando o PXE, a exemplo do *PelicanHPC*. O *Rocks Cluster* pode ser utilizado tanto com máquinas reais como virtuais.

Scyld ClusterWare [SCYLD 2012] é uma opção comercial para a criação e gerência de ambientes de *cluster*. Permite a execução em ambiente de *cluster* de programas com MPI e programas PVM (*Parallel Virtual Machine*), que é uma API para programação de aplicações paralelas. A criação dos nós é executada com o auxílio do PXE e o sistema *Scyld ClusterWare* possui uma interface Web para as tarefas de controle

do *cluster*.

O *PelicanHPC* e o *Rocks Cluster* se diferenciam do trabalho proposto na medida em que se limitam a criação do ambiente. Ambos os sistemas não auxiliam na utilização dos *clusters* criados e, principalmente, têm por objetivo a criação de um único ambiente de *cluster* por conjunto de máquinas, dedicando este ambiente a apenas um tipo de utilização.

O *Scyld ClusterWare* é, dos trabalhos abordados, o que mais se aproxima, quanto ao funcionamento do sistema proposto. Porém vale ressaltar que para seu uso é necessário dedicar toda a infraestrutura na qual pretende-se criar o ambiente de *cluster*. O presente trabalho se diferencia ao permitir que até mesmo máquinas que estejam sendo utilizadas para outros fins, como por exemplo os *desktops* de uso pessoal, sejam utilizados em um ou mais *clusters*. As máquinas já existentes em outros projetos podem ser incluídas sem que suas configurações sejam alteradas e uma máquina que esteja dedicada ao sistema proposto pode ser utilizada para outros fins quando necessário. Outra diferença entre o sistema criado e o *Scyld ClusterWare*, é que neste último a submissão é feita manualmente por meio de comandos em terminal, já o sistema criado é capaz de submeter e gerenciar as tarefas através da interface gráfica.

Apesar de sua relevância no cenário de HPC, a criação de ambientes de *clusters* e sua utilização não são tarefas simples. Este trabalho procura contribuir para esta tarefa ao fornecer uma ferramenta capaz de fornecer as funcionalidades apresentadas na próxima seção.

3. Funcionalidades e Arquitetura da Ferramenta

O sistema aqui apresentado visa atender a necessidade de criação de *clusters* sob demanda. O *cluster* criado é capaz de atender os requisitos específicos de cada pesquisador ou projeto, eliminando assim a necessidade destes adaptarem-se a um ambiente genérico. **A ferramenta Naegling está disponível para ser acessada através das instruções disponíveis em <http://comcidis.lncc.br/salaodeferramentas.php>.**

3.1. Visão Geral

O sistema proposto por este trabalho está baseado em três módulos principais: Programa Servidor; Interface Gráfica do Usuário - GUI (*Graphical User Interface*); e Rotinas do *Cluster*. Na figura 1 pode-se observar a arquitetura aqui apresentada com suas relações de troca de mensagens e transferência de arquivos.

O programa servidor, entre outros atributos, é encarregado de serviços como a criação e o controle das máquinas virtuais, o gerenciamento local dos *templates* dos *clusters* utilizados e pela comunicação entre o usuário e o *cluster* propriamente dito. Este é o módulo responsável por todo o trabalho de gerenciamento e criação dos *clusters* do sistema.

A GUI ou interface gráfica é o meio pelo qual o usuário irá interagir com o sistema. Através da GUI será possível criar os *clusters*, gerenciar os *templates* disponíveis, submeter os trabalhos a fim de serem processados, receber os resultados destes trabalhos e ainda a interface fornecerá um acesso gráfico ao ambiente de *clusters*, permitindo assim, uma forma para controlar diretamente o *cluster* e corrigir erros.

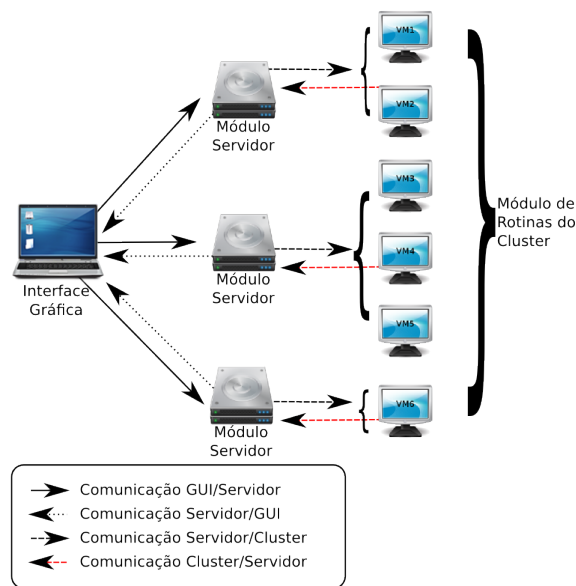


Figura 1. Arquitetura do Sistema Proposto

Para que o servidor seja capaz de executar os trabalhos nos *clusters*, é necessário o módulo de rotinas do *cluster*. Este é composto por funções e métodos que controlam o fluxo do trabalho sendo executado desde a configuração dos nós necessários ao processo de clusterização até o retorno dos resultados obtidos pelo processamento das tarefas. Através deste sistema o usuário será capaz de trabalhar com a maior quantidade possível de tipos de *clusters* diferentes, ao desenvolver um pacote de rotinas específicas para cada tipo suportado. O presente trabalho possui estas rotinas, inicialmente, implementadas para *clusters* de MPI, usando a implementação gratuita e de código aberto MPICH2.

3.2. Aplicação

O módulo servidor foi desenvolvido tendo em mente sua utilização em sistemas que atendam as especificações POSIX (*Portable Operating System Interface* - Interface Portável entre Sistemas Operacionais) em especial sistemas operacionais do tipo GNU/Linux. Isto é justificado pela dominância deste tipo de sistemas operacionais na área de computação de alto desempenho, sua segurança, código aberto, escalabilidade, desempenho, estabilidade e flexibilidade.

Apesar do módulo servidor estar focado em sistemas operacionais Linux, por ser a plataforma ideal para os *clusters*, nada impede que o controle seja feito em plataformas diferentes, nas quais o usuário esteja mais familiarizado ou tenha maior facilidade de acesso. Partindo deste princípio, a interface gráfica responsável pela interação entre o usuário e os *clusters*, pode ser instalada e utilizada em uma gama de sistemas diferentes, sendo a máquina virtual JavaTM o único pré-requisito para sua utilização.

3.3. Procedimentos Para a Criação de *Clusters*

Para iniciar o processo de criação do *cluster* é necessário apenas escolher um nome para o novo *cluster*. Ao ser criado, este inicialmente não possui nenhum nó. O passo seguinte é a criação dos nós que são divididos em dois tipos básicos: *Master nodes*

(nós mestres) e *working nodes* (nós de trabalho). Um *cluster* neste sistema é composto por um nó mestre e n ($n \geq 0$) nós de trabalho.

Para a criação de um nó do *cluster*, seja ele o nó mestre ou os nós de trabalho, é necessário primeiramente que haja pelo menos uma máquina com o módulo servidor. As máquinas que possuem este módulo são denominadas de *hosts* ou hospedeiras. Na figura 2 está representado o fluxo necessário para a criação de um nó do *cluster*.

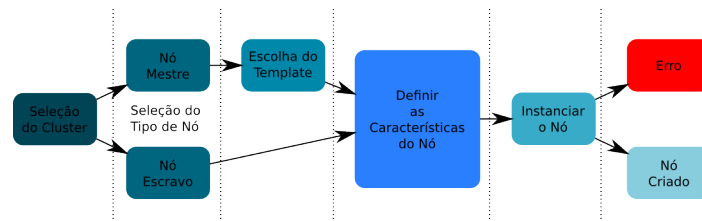


Figura 2. Fluxograma da Criação de um Nó do *Cluster*

O nó mestre é o encarregado do gerenciamento interno do *cluster*, divisão e submissão do trabalho que será executado e pela consolidação dos resultados. No caso do sistema aqui proposto, o nó mestre é a máquina do *cluster* que possui instalado o módulo de rotinas do sistema. Para cada tipo de *cluster* existe um módulo diferenciado.

Uma vez selecionado *Master* como sendo o tipo de nó, o usuário terá que selecionar o *template* do *cluster*. O *template* é um modelo que será disponibilizado, ou poderá ser criado e personalizado. No escopo deste trabalho foi criado um *template* para *clusters* do tipo MPI utilizando o MPICH2. No sistema aqui proposto os nós de trabalho são máquinas virtuais *diskless*, que não possuem disco de armazenamento permanente.

3.4. Procedimentos Para a Submissão de Tarefas

As tarefas (*jobs*) são criadas para um determinado *cluster* através do botão para criação de *jobs* na barra de ferramentas da interface gráfica. Cada tarefa é composta de pelo menos um arquivo (fonte ou binário), podendo anexar quantos arquivos auxiliares forem necessários à correta execução do trabalho, e um *script* de execução. Após escolher a opção de criação de *jobs* na barra de ferramentas, aparecerá uma janela na qual é possível nomear e escolher os arquivos da tarefa. Ao terminar de escolher os arquivos necessários uma segunda janela permitirá a criação do *script* de execução de forma iterativa, sendo possível escolher através de menus como a tarefa será executada. Caso seja necessário o usuário pode editar manualmente o *script* nesta mesma janela. As figuras 3(a) e 3(b) mostram as janelas da interface gráfica responsáveis pela criação de um *Job*.

Através da janela de visualização de *jobs*, figura 4, é possível gerenciar a tarefa. Nela é possível enviar os arquivos de um determinado *job* para as máquinas do *cluster*, iniciar a execução da tarefa e recuperar os resultados do processamento.

Para executar uma determinada tarefa, os arquivos têm que estar nas máquinas do *cluster*. A transferência é realizada por um procedimento de cópia e redirecionamento desenvolvida especificamente para isto. É possível acompanhar o andamento da execução por meio do acesso gráfico fornecido. Ao término pode-se ainda realizar o *download* dos resultados através da barra de ferramentas. Na figura 5 é apresentado o processo para

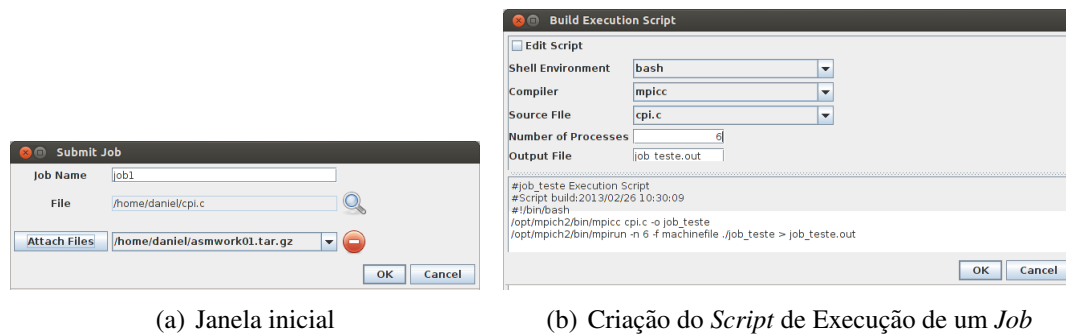


Figura 3. Janelas do processo de criação de *Job*

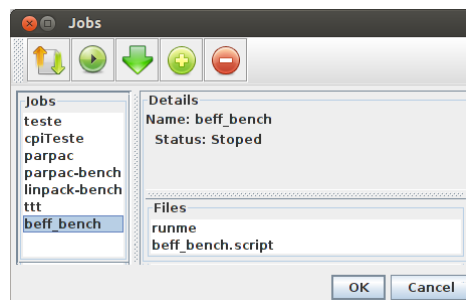


Figura 4. Janela Para Visualização e Execução de um *Job*

a criação, submissão, execução e reexecução de um determinado *job* de um *cluster*. O módulo do servidor funciona como a interface de comunicação entre o nó mestre de um *cluster* e a interface gráfica acessada pelo usuário, por isso ele efetua troca de mensagens tanto entre o módulo de rotinas do *cluster* como com o módulo da interface gráfica, para isso foi desenvolvido um protocolo próprio.

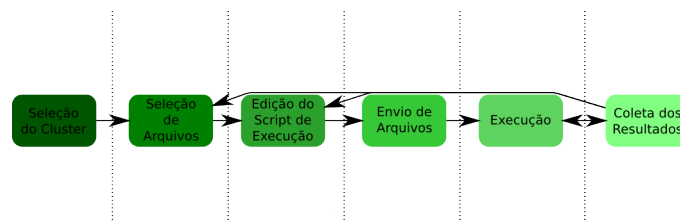


Figura 5. Fluxograma que Descreve a Criação e Submissão de um *Job*

4. Componentes e Funcionalidades da GUI

O módulo da interface gráfica está desenvolvido em JavaTM para viabilizar sua portabilidade e facilidade de execução. Este é o módulo responsável pela persistência dos dados do usuário, pela criação de *clusters* pelos usuários, pela submissão de tarefas, pelo acesso gráfico e pela interação do usuário com o sistema. Na figura 6 é exibida a janela principal da interface gráfica, através da qual o usuário realiza todas as operações necessárias ao controle do *cluster* e a submissão e o controle das tarefas.

Este módulo pode ser dividido de acordo com suas funcionalidades em cinco partes principais: controle do *cluster*, submissão de trabalhos, comunicação entre a interface e o servidor, acesso gráfico ao ambiente dos *clusters* e persistência de dados.

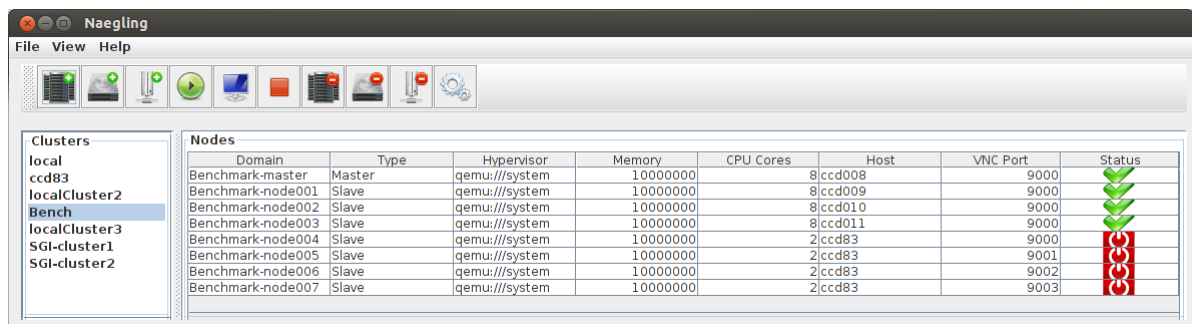


Figura 6. Janela Principal da Interface Gráfica

1. Controle do *Cluster* - É responsável por permitir ao usuário criar o *cluster* propriamente dito bem como seus nós, ligá-los e verificar o estado de cada *cluster*. Para facilitar estas tarefas foram criados menus, barras de ferramentas e janelas, tornando o processo mais intuitivo.
2. Acesso ao Ambiente - Este sistema foi desenvolvido de forma que o usuário não tenha que acessar o ambiente de *cluster* da forma tradicional, que é geralmente feito por acesso remoto via SSH (*Secure Shell*). Todo o processo, de criação do *cluster*, submissão de tarefas e verificação do resultado, foi desenvolvido de forma que possa ser executado diretamente através da interface gráfica.

Caso o usuário do sistema tenha necessidade de acessar o ambiente de alto desempenho, seja por motivo de um erro indeterminado que esteja ocorrendo ou ele queira acompanhar o processamento do trabalho, a interface gráfica possui esta capacidade através do cliente VNC. A figura 8 é uma captura de tela realizada durante um acesso gráfico ao ambiente de um *cluster*.

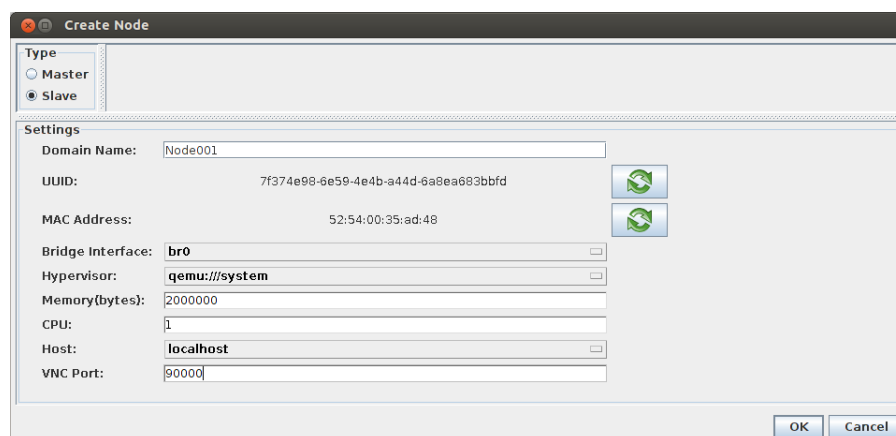


Figura 7. Janela de Criação de Nó

5. Conclusões e Trabalhos Futuros

O presente trabalho foi desenvolvido para suprir uma lacuna existente no atual cenário de HPC, facilitando tanto a criação de *clusters* sob demanda, permitindo que o mesmo seja efetuado a partir de uma interface gráfica, sem complicações para o usuário final, como também, auxiliar o usuário durante todo o processo de submissões de tarefas, acompanhamento do trabalho e recuperação de resultados.

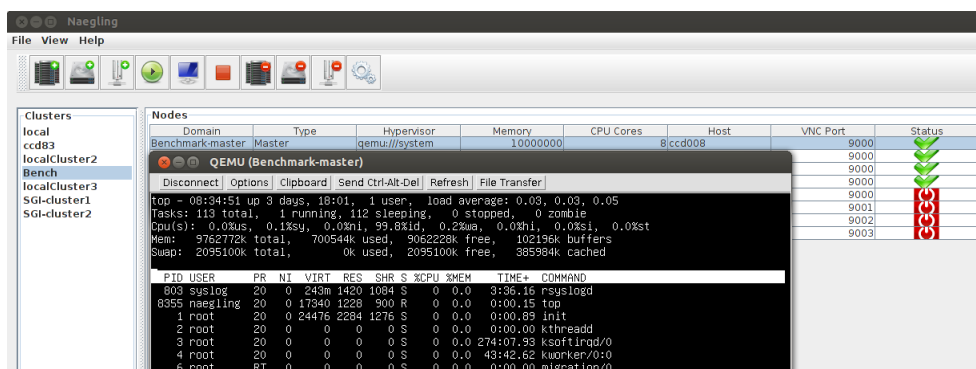


Figura 8. Acesso Gráfico ao Ambiente

Com o sistema apresentado, atualmente é possível criar *clusters* de centenas de nós em questão de minutos, enquanto normalmente esta tarefa demoraria dias, semanas ou até meses. Além disto, todo o processo de submissão de tarefas também foi simplificado, podendo ser realizado através de uma interface gráfica instalada no sistema operacional de preferência do usuário do *cluster*. O sistema possibilita que todo o acesso seja realizado através da interface criada, não mais necessitando acesso remoto para realizar a tarefa.

Apesar destas melhorias fornecidas, ainda há melhorias e capacidades a serem adicionadas. A seguir será listado alguns itens importantes que serão estudados e implementados:

- Ampliar os tipos de *Templates* suportados pelo sistema.
- Ampliar as capacidades de criações dos nós (criação de máquinas virtuais com discos, criação dos nós em máquinas reais).
- Ampliar os tipos de *Hypervisors* suportados.
- Aperfeiçoar a interface de submissão de tarefas para novos modelos de aplicações.
- Implementar um modelo de segurança em cada nível do ambiente.
- Portar a interface gráfica, de uma aplicação tipo *desktops*, para um ambiente *web*.

Agradecimentos

Os autores agradecem o apoio recebido do Conselho Nacional de Desenvolvimento Científico e Tecnológico - CNPq, por meio do projeto - Programa de Capacitação Institucional - LNCC/MCT.

Referências

- [Pacheco 2011] Pacheco, P. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1st edition.
- [PELICANHPC 2012] PELICANHPC (2012). Pelicanhpc: A gnu/linux distribution to create a hpc cluster for mpi based parallel computing. [Online; acessado em 05-fevereiro-2014 URL:<http://pareto.uab.es/mcreel/PelicanHPC/>].
- [ROCKSCLUSTER 2012] ROCKSCLUSTER (2012). www.rocksclusters.org — rocks website. [Online; acessado em 05-fevereiro-2014 URL:<http://www.rocksclusters.org/wordpress/>].
- [SCYLD 2012] SCYLD (2012). Scyld clusterware hpc. [Online; acessado em 05-fevereiro-2014 URL:<http://gilgamesh.cheme.cmu.edu/scyld-doc/users-guide/users.html>].